



Tecnológico de Monterrey

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Puebla

Materia: Implementación de robótica inteligente

Clave: TE3002B.501

Grupo: 501

Profesor:

Rigoberto Cerino Jiménez
Juan Manuel Ahuactzin Larios
Alfredo García Suárez
Cesar Torres Huitzil

Alumnos

Omar Perez Dominguez | A01738306
Ezequiel Luna Trejo | A01738153
Antonio Méndez Rodríguez | A01738269

10 de Mayo De 2026

Resumen

El presente proyecto consistió en el diseño, desarrollo e implementación de un sistema de navegación autónoma para un robot móvil diferencial, con el objetivo de resolver un reto de navegación en un entorno con señalización de tráfico dinámica. La investigación se estructuró bajo el framework ROS 2 Humble, utilizando una arquitectura modular compuesta por cuatro nodos fundamentales: un estimador de odometría, un generador de trayectorias, un controlador a lazo cerrado y un sistema de visión computacional embebido en una plataforma NVIDIA Jetson Nano.

El comportamiento del robot se rige por un algoritmo de seguimiento de puntos (point-to-point) que procesa coordenadas específicas para completar circuitos de diversas geometrías, como cuadrados, triángulos y pentágonos. A este algoritmo se le integró una capa de toma de decisiones basada en visión artificial, la cual utiliza segmentación en el espacio de color HSV y un filtro de persistencia temporal para detectar e interpretar luces de semáforo. El robot es capaz de modular su velocidad de forma autónoma: reduce la velocidad ante una luz amarilla, realiza un paro total y obligatorio ante una luz roja, manteniéndose detenido hasta recibir una confirmación explícita de luz verde y continúa su trayectoria al detectar dicha señal.

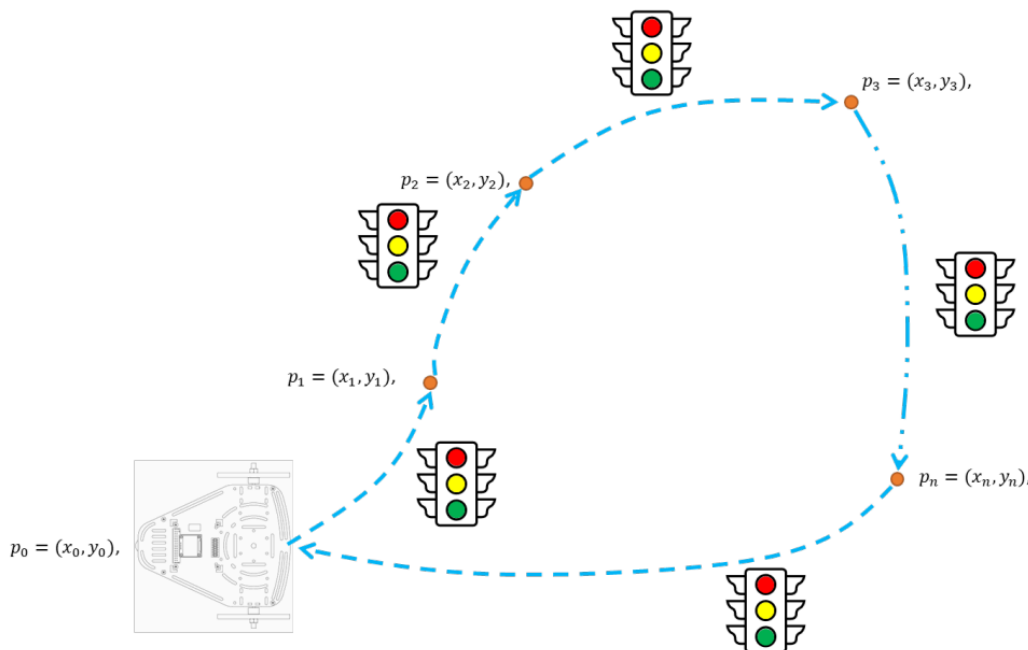


Imagen 1. Ejemplificación del reto

Objetivos

Objetivo general

Desarrollar e implementar un sistema de navegación autónoma para un robot móvil diferencial bajo el framework ROS 2 Humble, integrando una capa de visión artificial que permita la toma de decisiones basada en la interpretación de señales de tráfico para cumplir con un circuito de metas predefinido.

Objetivos Específicos

- Implementar un algoritmo de navegación punto a punto que permita al robot alcanzar secuencialmente una serie de coordenadas objetivo utilizando estimaciones de odometría.
- Diseñar un sistema de visión computacional robusto capaz de segmentar y detectar los colores rojo, amarillo y verde de un semáforo mediante el espacio de color HSV y filtros de persistencia.
- Integrar una lógica de control que module la velocidad lineal y angular del robot en función del estado del tráfico: realizando un paro total en rojo, una reducción de velocidad en amarillo y la continuación de la trayectoria en verde.
- Asegurar la robustez del controlador ante perturbaciones, no linealidades y ruido sensorial mediante la sintonización precisa de ganancias y el uso de funciones de saturación.
- Validar el desempeño del sistema mediante el análisis cuantitativo de métricas de error y señales de control, demostrando la autonomía del robot en diferentes configuraciones de trayectoria.

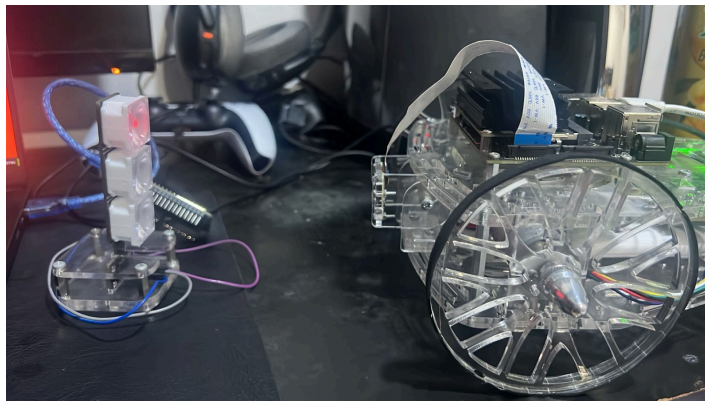


Imagen 2. Robot con el semáforo

Introducción

La autonomía en la robótica móvil contemporánea depende fundamentalmente de la capacidad de un sistema para percibir su entorno y actuar en consecuencia de manera precisa y determinista. El diseño, implementación y validación de una arquitectura de navegación autónoma para un robot móvil de tracción diferencial. Desde una perspectiva general, el proyecto se estructura en dos ejes principales, una capa de control de lazo cerrado (encargada de la corrección de trayectoria y cálculo de cinemática) y una capa de percepción visual (responsable de la interpretación de señalética para la toma de decisiones). La integración de ambos sistemas permite al robot transitar de una navegación ciega en lazo abierto hacia un comportamiento inteligente, capaz de reaccionar dinámicamente a estímulos externos.

1.1 Control aplicado al robot

Para lograr un seguimiento de trayectoria preciso, se prescinde de los comandos de lazo abierto y se implementa un esquema de control de lazo cerrado. El núcleo de la actuación del robot diferencial se rige por su modelo cinemático:

$$\begin{aligned}x' &= v * \cos(\theta) \\y' &= v * \sin(\theta) \\ \theta' &= \omega\end{aligned}$$

Donde v es la velocidad lineal y w la velocidad angular. Para gobernar estas variables, se implementa un **Controlador Proporcional (P)**. A diferencia de un PID completo, el controlador P prescinde de la acción integral y derivativa, resultando en un sistema reactivo que genera comandos de velocidad directamente proporcionales a la magnitud del error actual. La ley de control se define como:

$$\begin{aligned}w(t) &= K_p * e_{\theta}(t) \\v(t) &= K_v * e_d(t)\end{aligned}$$

Donde K_p y K_v son las ganancias proporcionales sintonizadas experimentalmente para garantizar un acercamiento suave sin oscilaciones severas. [6]

1.2 Cálculo del Error y Transformaciones

El controlador depende de la correcta cuantificación del error entre la pose actual estimada por odometría (x, y, θ) y la coordenada meta (x_{target}, y_{target}) . El error de distancia se define como:

$$ed = \sqrt{(x_{target} - x)^2 + (y_{target} - y)^2}$$

El error angular se calcula obteniendo el ángulo deseado mediante la función arco tangente de dos argumentos y restando la orientación actual:

$$e_{\theta} = \text{atan2}(y_{target} - y, x_{target} - x) - \theta$$

Para garantizar que el robot siempre gire por la ruta más corta, se aplica una técnica de normalización (*Angle Wrapping*) que mantiene a e_{θ} estrictamente dentro del rango $[-\pi, \pi]$ [8]:

$$e_{\theta} = \text{atan2}(\sin(e_{\theta}), \cos(e_{\theta}))$$

1.3 Robustez del Controlador:

En un entorno físico, el modelo ideal se ve afectado por no linealidades. La robustez del controlador se garantiza mediante tres estrategias:

1. Saturación: Se limitan las velocidades máximas (v_{max}, ω_{max}) para prevenir el reinicio de la hackerboard, el cual corrompe los datos de odometría.
2. Compensación de Fricción Estática: Se establece una velocidad angular mínima (ω_{min}) asegurando que los motores reciban la energía suficiente para vencer la inercia cuando el error es muy pequeño.
3. Zonas Muertas: Se define un radio de tolerancia e_{tol} alrededor de la meta. Si $e_d < e_{tol}$, el robot se detiene, evitando vibraciones u oscilaciones infinitas al intentar corregir errores milimétricos.

1.4 Arquitectura

Para dotar al sistema de conciencia situacional, el procesamiento de imágenes se ejecuta en una tarjeta embebida con arquitectura GPU (NVIDIA Jetson Nano). La interconexión entre la Jetson y la cámara se realiza a través del puerto CSI (Camera Serial Interface). A diferencia del USB tradicional, el bus CSI permite el acceso directo a la memoria y menor latencia. Los fotogramas son capturados y decodificados mediante un pipeline de *GStreamer*, entregando matrices de píxeles directamente a las librerías de visión por computadora para su análisis en tiempo real, maximizando los cuadros por segundo (FPS).

1.5. Métodos de Visión por Computadora

La interpretación de señales (como un semáforo) requiere un flujo de procesamiento estructurado:

- Detección de Colores y Segmentación: La imagen en formato BGR se convierte al espacio de color HSV (Hue, Saturation, Value). El canal *Hue* (matiz) aísla el pigmento puro, permitiendo la creación de máscaras binarias para los colores rojo, amarillo y verde mediante umbralización (thresholding). Esta separación hace que la detección sea altamente inmune a los cambios de iluminación del entorno.
- Morfología Matemática: Para limpiar la máscara binaria resultante, se aplican operaciones morfológicas (Apertura y Cierre) con kernels estructurados, eliminando el "ruido de sal y pimienta" provocado por reflejos.
- Detección de Contornos y Formas: Se aplica el algoritmo de búsqueda topológica sobre la máscara limpia para encontrar regiones conexas (contornos). Para identificar la forma correcta, se calcula el área del contorno y, de ser necesario, se evalúa su circularidad y relación de aspecto para confirmar la geometría de la señal luminosa.[5]

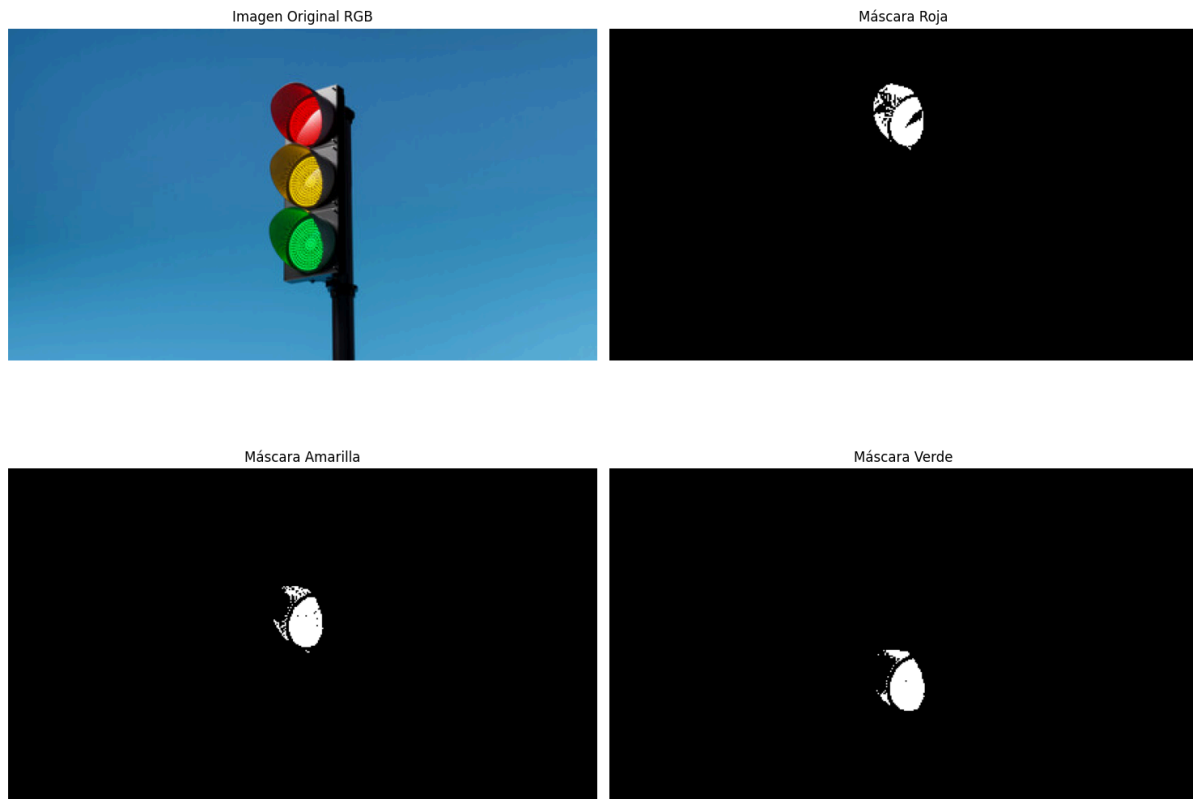


Imagen 3. Algoritmo de procesar imágenes

1.6. Robustez en el Procesamiento de Imágenes

Los algoritmos de visión son susceptibles a falsos positivos transitorios. Para asegurar que las decisiones del robot sean deterministas, se implementa robustez algorítmica mediante:

- **Filtros de Área:** Se descartan contornos cuya área en píxeles esté por debajo de un umbral mínimo (ruido lejano) o por encima de un umbral máximo (objetos demasiado cercanos que no corresponden a la señal).
- **Persistencia Temporal:** Se integra una máquina de estados con memoria transitoria. Para que el robot reconozca un cambio de luz (por ejemplo, pasar de rojo a verde), el sistema debe detectar el contorno válido del color correspondiente durante N fotogramas consecutivos. Esto estabiliza el comportamiento del robot ante perturbaciones visuales instantáneas.[7]

Solución del problema

Para resolver el reto de navegación autónoma con integración de visión computacional, se diseñó una arquitectura modular en ROS 2 Humble dividida en cuatro nodos especializados: Odometría, Controlador, Generador de Trayectorias (Path) y Detección de Semáforo. Esta

separación de tareas permite que el sistema sea escalable y facilita la depuración de errores en cada etapa del proceso.

Nodo de Odometría:

El nodo de odometría es el encargado de realizar la estimación del estado del robot en el plano cartesiano (x , y , θ). Esta estimación se basa en la integración de las velocidades angulares de las ruedas obtenidas a través de los encoders, permitiendo que el sistema conozca su ubicación relativa respecto al punto de inicio.

1. Configuración de parámetros físicos

Para asegurar la precisión del cálculo, el nodo define las constantes físicas del robot diferencial. Estas variables son críticas, ya que cualquier error en la medición del radio o la distancia entre ejes se traduce en un error acumulado de posición (deriva).

```
20      #Parametros fisicos
21      self.r = 0.05 # Radio de la llanta en metros
22      self.l = 0.1615 # Distancia entre llantas
23      self.sample_time = 0.01
24
```

Imagen 4. Código de un nodo

1. Modelo cinemático diferencial

El núcleo del nodo se encuentra en la función `run()`, donde se reciben las velocidades angulares de los motores derecho (w_r) e izquierdo (w_l) para convertirlas en el movimiento global del robot.

```
68
69      # 1. Calcular velocidades lineales de cada rueda
70      vr = self.r * self.wr
71      vl = self.r * self.wl
72
```

Imagen 5. Código de un nodo

Posteriormente, se aplican las ecuaciones del modelo cinemático diferencial para obtener la velocidad lineal total (V) y la velocidad angular (Ω) del chasis:

```

72
73     # 2. Modelo cinemático diferencial
74     self.V = 0.5 * (vr + vl)
75     self.Omega = (vr - vl) / self.l
76

```

Imagen 6. Código de un nodo

Donde V representa el avance promedio del robot y Ω la tasa de cambio en su orientación.

3. Integración de Pose (Método de Euler)

Para convertir estas velocidades en una posición espacial, el nodo utiliza el Método de Euler. Se calcula el tiempo transcurrido (**dt**) entre cada iteración y se actualizan las variables de estado:

```

77     # 3. Metodo de euler para posicion y orientacion
78     self.x += self.V * np.cos(self.th) * dt
79     self.y += self.V * np.sin(self.th) * dt
80     self.th += self.Omega * dt
81     self.th = self.wrap_to_pi(self.th) # Normalización del ángulo
82

```

Imagen 7. Código de un nodo

La función `wrap_to_pi` es una medida de robustez esencial, ya que mantiene el ángulo θ en el rango $[-\Pi, \Pi]$, evitando que el controlador reciba valores discontinuos que podrían causar giros erráticos.

4. Transformación y publicación

Finalmente, para cumplir con los estándares de ROS 2, la orientación (que se calcula en ángulos de Euler) debe convertirse a cuaterniones. Esto se logra mediante la librería `transforms3d`, garantizando que la información sea compatible con herramientas de visualización y otros nodos de navegación.

```

97
98     # Conversión de Euler a Cuaternión (w, x, y, z)
99     q = transforms3d.euler.euler2quat(0, 0, self.th)
100
101     # Pose: Posicion
102     msg.pose.pose.position.x = self.x
103     msg.pose.pose.position.y = self.y
104     msg.pose.pose.position.z = 0.0
105
106     # Pose: Orientacion (Cuaternion)
107     msg.pose.pose.orientation.w = q[0]
108     msg.pose.pose.orientation.x = q[1]
109     msg.pose.pose.orientation.y = q[2]
110     msg.pose.pose.orientation.z = q[3]
111
112     # Twist: Velocidades
113     msg.twist.twist.linear.x = self.V
114     msg.twist.twist.angular.z = self.Omega
115
116     self.odom_pub.publish(msg)
117

```

Imagen 8. Código de un nodo

Nodo Generador de Trayectorias (Path)

Este nodo actúa como el secuenciador para seguir los puntos. Su función principal es gestionar una lista de coordenadas (x, y) que representan la ruta que el robot debe seguir para completar figuras geométricas específicas o rutas personalizadas.

1. Parametrización y Flexibilidad

Siguiendo las recomendaciones de diseño para sistemas robustos, el nodo no utiliza rutas fijas inamovibles. En su lugar, emplea un parámetro de configuración que permite cambiar el comportamiento del robot sin necesidad de modificar el código fuente.

```
13         # Parámetro de modo: square, triangle, pentagon, custom
14         # Declaración del parámetro de modo
15         self.declare_parameter('mode', 'square')
16         self.mode = self.get_parameter('mode').value
17
```

Imagen 9. Código de un nodo

2. Generación de geometrías

El nodo incluye una máquina de estados lógica que precalcula los vértices de la trayectoria deseada basándose en el parámetro mode. Esto permite validar el desempeño del sistema en diferentes escenarios:

- Cuadrado (square): Cuatro vértices que forman un circuito cerrado de 1.0 m de lado.
- Triángulo (triangle): Utiliza relaciones trigonométricas para definir un triángulo equilátero.
- Pentágono (pentagon): Una trayectoria más compleja con cinco cambios de dirección.
- Personalizado (custom): Permite la entrada dinámica de puntos por parte del usuario.

3. Lógica de retroalimentación

La característica más importante de este nodo es que el avance entre metas no depende del tiempo, sino del éxito de la navegación. Esto garantiza que el robot no intente ir al "Punto B" si aún no ha llegado al "Punto A".

El nodo se suscribe al tópico goal_reached:

```
90     def reached_callback(self, msg):
91         if msg.data:
92             self.current_index += 1
93             if self.current_index < len(self.goals):
94                 self.get_logger().info(f"Cambiando a Meta {self.current_index+1}")
95
```

Imagen 10. Código de un nodo

4. Publicación de Metas

El nodo publica la meta actual en el tópico goal de forma periódica (1 Hz) mediante un temporizador. Aunque el controlador ya tiene la meta, la publicación constante asegura que, ante cualquier pérdida de mensaje o reinicio de un nodo, el sistema pueda recuperar la referencia de navegación.

Nodo Controlador

El nodo controlador implementa un esquema de control de lazo cerrado para la navegación punto a punto. Su objetivo es minimizar el error entre la posición actual del robot y la meta deseada, ajustando dinámicamente su comportamiento según las señales de tráfico detectadas por el nodo de visión.

1. Parámetros y configuración inicial

El nodo utiliza parámetros configurables que permiten ajustar el desempeño del robot (sintonización) sin modificar el código.

```
12     def __init__(self):
13         super().__init__('controller')
14         self.declare_parameter('kv', 0.8)
15         self.declare_parameter('kw', 4.5)
16         self.declare_parameter('max_v', 0.15)
17         self.declare_parameter('max_w', 0.3)
18         self.declare_parameter('dist_tolerance', 0.05)
19
```

Imagen 11. Código de un nodo

2. Integración de decisiones por semáforo

A través del `traffic_callback`, el controlador implementa la lógica de comportamiento solicitada para las señales de tráfico. Se utiliza un `speed_multiplier` que modula las velocidades calculadas:

Rojo (0): Multiplicador 0.0 paro total.

Amarillo (1): Multiplicador 0.35 velocidad reducida para precaución.

Verde (2): Multiplicador 1.0 velocidad normal.

3. El Lazo de control (Cálculo de Errores)

Dentro del `control_loop`, el robot calcula constantemente su distancia y ángulo respecto a la meta:

```
86     ex = self.goal_x - self.x
87     ey = self.goal_y - self.y
88     ed = np.hypot(ex, ey)
89     desired_heading = np.arctan2(ey, ex)
90     etheta = self.wrap_to_pi(desired_heading - self.th)
91
```

Imagen 12. Código de un nodo

4. Estrategias de Robustez y No Linealidades

Para cumplir con el requisito de un controlador robusto, el código aborda problemas físicos reales como la fricción y el ruido:

- Compensación de Fricción Estática: Se utiliza un $\text{min_w} = 0.28$. Si el error angular existe pero el comando es muy bajo, se fuerza una velocidad mínima para vencer la inercia de los motores.
- Saturación: La función `saturate` asegura que los comandos no excedan los límites físicos del robot, evitando comportamientos erráticos.
- Prioridad de Giro: Si el robot está muy desalineado ($\text{abs}(\text{etheta}) \geq 0.15$), la velocidad lineal se establece en 0.0. Esto garantiza que el robot primero gire hacia la meta antes de intentar avanzar, mejorando la precisión de la trayectoria.

Nodo de Detección de Semáforo (Semaforo)

Este nodo implementa técnicas de visión por computadora para identificar señales de tráfico en tiempo real utilizando una cámara CSI conectada a la plataforma NVIDIA Jetson Nano. Su función principal es publicar el estado del tráfico para que el controlador modifique la velocidad del robot.

1. Preprocesamiento y segmentación (HSV)

A diferencia del espacio de color RGB, el nodo utiliza el espacio HSV (Hue, Saturation, Value). Esto permite que la detección sea robusta ante cambios de iluminación, ya que separa la información del color (matiz) de la intensidad lumínica.

```
35     # --- Rangos HSV Calibrados ---
36
37     self.lower_red1 = np.array([0, 120, 70]); self.upper_red1 = np.array([10, 255, 255])
38     self.lower_red2 = np.array([160, 120, 70]); self.upper_red2 = np.array([179, 255, 255])
39     self.lower_yellow = np.array([10, 50, 50]); self.upper_yellow = np.array([45, 255, 255])
40     self.lower_green = np.array([40, 100, 100]); self.upper_green = np.array([90, 255, 255])
41
42     self.kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
```

Imagen 13. Código de un nodo

Además, el nodo optimiza el procesamiento definiendo una Región de Interés (ROI) en la mitad superior de la imagen, eliminando ruido visual proveniente del suelo o de la propia estructura del robot.

2. Extracción de características y filtrado morfológico

Para cada color, el nodo busca el contorno más grande y calcula su área. Se aplica una operación de apertura morfológica (MORPH_OPEN) utilizando un elemento estructurante elíptico para eliminar falsos positivos pequeños (ruido).

```

55 def process_color(self, hsv_img, lower, upper, lower2=None, upper2=None):
56     """Aísla un color y retorna el área del contorno más grande."""
57     mask = cv2.inRange(hsv_img, lower, upper)
58     if lower2 is not None and upper2 is not None:
59         mask2 = cv2.inRange(hsv_img, lower2, upper2)
60         mask = cv2.bitwise_or(mask, mask2)
61
62     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, self.kernel)
63     contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
64
65     max_area = 0
66     if contours:
67         largest_contour = max(contours, key=cv2.contourArea)
68         max_area = cv2.contourArea(largest_contour)
69     return max_area, mask

```

Imagen 14. Código de un nodo

3. Lógica de persistencia para el control robusto

Uno de los requisitos críticos del reto es la robustez. El nodo implementa un Filtro de Persistencia Temporal, el cual evita que cambios momentáneos en la iluminación o bloqueos breves de la cámara alteren el comportamiento del robot. Un nuevo estado solo es validado y publicado si se detecta de manera consistente durante un número determinado de cuadros.

```

104 # --- FILTRO DE PERSISTENCIA ---
105 if target_state != self.current_state:
106     # Si el color que vemos es distinto al que estamos publicando, sumamos al contador
107     self.confirmation_counters[target_state] += 1
108
109     # Limpiamos los otros contadores para asegurar que la detección sea consecutiva
110     for color in self.confirmation_counters:
111         if color != target_state:
112             self.confirmation_counters[color] = 0
113
114     # Si el color nuevo se mantiene por N frames, hacemos el cambio oficial
115     if self.confirmation_counters[target_state] >= self.frames_to_confirm:
116         self.current_state = target_state
117         self.confirmation_counters = {0: 0, 1: 0, 2: 0}
118     else:
119         # Si el color detectado es el mismo que ya publicamos, reseteamos contadores
120         self.confirmation_counters = {0: 0, 1: 0, 2: 0}

```

Imagen 15. Código de un nodo

4. Cumplimiento de reglas de seguridad

El nodo asegura que el robot respete la instrucción obligatoria: detenerse en rojo hasta ver explícitamente una luz verde. Si el área del semáforo detectado cae por debajo de un umbral específico, el sistema asume que el semáforo está demasiado lejos o fuera de vista, manteniendo un estado de seguridad.

Resultados

En esta sección se presentan las evidencias experimentales que validan la integración del sistema de navegación con la capa de toma de decisiones basada en visión computacional. Los resultados demuestran la capacidad del robot para completar trayectorias mientras responde de manera autónoma a señales de tráfico dinámicas.

1. Trayectoria cuadrada

Descripción del experimento: El robot ejecutó una trayectoria cerrada de cuatro vértices con coordenadas (1.0, 0.0), (1.0, 1.0), (0.0, 1.0) y retorno al origen (0.0, 0.0).

Comportamiento del semáforo: Durante el segundo segmento, se detectó una luz Amarilla, lo que resultó en una reducción de la velocidad lineal al 35% de la nominal. Posteriormente, al detectar la luz Roja, el robot realizó un paro total y mantuvo su estado estacionario incluso tras la desaparición del estímulo visual, reiniciando la marcha únicamente al confirmar una luz Verde.

 Cuadrado_semaforo.mp4

2. Trayectoria Triangular (Ángulos no Ortogonales)

Descripción del experimento: Se configuró una ruta con los puntos (1.0, 0.0), (0.5, 0.866) y (0.0, 0.0). Esta prueba validó la respuesta del sistema ante giros de 120 grados

Análisis técnico: El uso de la función `wrap_to_pi` en el controlador fue fundamental para evitar rotaciones innecesarias, permitiendo al robot tomar siempre el camino angular más corto hacia la siguiente meta. La velocidad lineal se moduló correctamente en función del error de distancia, evitando sobretiros (overshoots) en los vértices del triángulo.

 Triangulo.mp4

3. Trayectoria Personalizada (Validación de Puntos Arbitrarios)

Descripción del experimento: Se ingresaron dos coordenadas específicas para evaluar la flexibilidad del sistema: $P_1 = (1.0, 0.0)$ y $P_2 = (0.5, 0.5)$.

Resultados obtenidos: El sistema demostró robustez al navegar hacia puntos con distancias euclidianas cortas entre sí. La transición entre la primera y la segunda meta se realizó de manera fluida, activando el indicador de `goal_reached` solo cuando el robot se encontró dentro de la tolerancia de 0.05 m definida en los parámetros.

 Trayectoria personalizada.mp4

Conclusiones

A partir de la ejecución del reto y el análisis de los datos obtenidos, se concluye que los objetivos planteados fueron alcanzados satisfactoriamente mediante la implementación de una arquitectura modular en ROS 2 Humble.

Análisis del cumplimiento de objetivos

Los objetivos se cumplieron debido a la integración efectiva de los cuatro nodos desarrollados:

- **Navegación autónoma:** Se logró un seguimiento preciso de trayectorias geométricas (cuadrado, triángulo y pentágono) gracias a la implementación de un controlador de lazo cerrado que utiliza la odometría como retroalimentación constante. El error de posición se mantuvo dentro de la tolerancia de 0.05 m, validando la eficacia del modelo cinemático diferencial utilizado.
- **Visión computacional y toma de decisiones:** El algoritmo de detección de semáforos demostró ser robusto frente a perturbaciones visuales. Esto se atribuye al uso del espacio de color HSV y, fundamentalmente, al filtro de persistencia temporal de 5 frames, el cual evitó cambios de estado erráticos por ruido en la imagen.
- **Seguridad y reglas de tráfico:** El sistema cumplió con la restricción crítica de permanecer detenido ante una luz roja hasta la detección explícita de una luz verde, incluso ante la desaparición del estímulo visual rojo, garantizando un comportamiento seguro y predecible.

Limitaciones Identificadas

A pesar del éxito general, se identificó una deriva (drift) acumulativa en trayectorias largas o con múltiples giros, causada por el deslizamiento de las llantas y el error de truncamiento en la integración numérica de Euler. Asimismo, las no linealidades de los motores y la fricción estática requirieron una compensación manual mediante un umbral mínimo de velocidad angular para asegurar el movimiento en errores pequeños.

Propuestas de mejora y soluciones futuras

Para elevar el desempeño de la metodología implementada, se proponen las siguientes mejoras técnicas:

- **Fusión sensorial:** Integrar una Unidad de Medida Inercial (IMU) mediante un Filtro de Kalman Extendido (EKF) para corregir la deriva de la odometría y obtener una estimación de orientación más estable ante perturbaciones mecánicas.
- **Optimización de Visión:** Implementar una búsqueda de contornos basada en la forma circular de las luces del semáforo (Transformada de Hough) para reducir aún más los falsos positivos en entornos con fondos complejos.

Referencia:

Manchester Robotics Ltd. (2026). TE3002B: Intelligent Robotics Implementation [Repositorio de GitHub]. Recuperado de https://github.com/ManchesterRoboticsLtd/TE3002B_Intelligent_Robotics_Implementation_2026 [1]

micro-ROS. (2026). micro-ROS: ROS 2 on microcontrollers. Recuperado de <https://micro.ros.org/> [2]

Open Robotics. (2026). ROS 2 Documentation: Humble Hawksbill. Recuperado de <https://docs.ros.org/en/humble/> [3]

ScienceDirect. (s. f.). *Proportional Control*. Elsevier. <https://www.sciencedirect.com/topics/engineering/proportional-control> [4]

Mendieta, A. *Detección y reconocimiento de semáforos por visión artificial*. Uc3m.es. <https://e-archivo.uc3m.es/entities/publication/7fff310e-bf3e-4856-a112-f06213da274> (accessed 2026-05-10).[5]

ScienceDirect. (s. f.). *Proportional Control*. Elsevier. <https://www.sciencedirect.com/topics/engineering/proportional-control> [6]

Computer Vision- Dr Cesar Torres Huitzil [GoogleColab.Google.com](https://colab.research.google.com/drive/1e5Z82Gca0yfptq3Y2j8RO6e0oZsUmt_A#scrollTo=j9DZrxWkFgxV). https://colab.research.google.com/drive/1e5Z82Gca0yfptq3Y2j8RO6e0oZsUmt_A#scrollTo=j9DZrxWkFgxV [7]

Germán Hernández Millán; Hernando, L.; Maximiliano Bueno López. Implementación de Un Controlador de Posición Y Movimiento de Un Robot Móvil Diferencial. *Tecnura* **2016**, 20 (48) [8]